Proyecto Parcial 1: Desarrollo de una FFT de 1024 puntos mediante mariposas

Procesamiento Digital de Señales

Profesor: Dr. Oscar Francisco Fuentes Casarrubias
Equipo 2:
Gonzalez Arredondo Rafael 319026489
Jiménez Gutiérrez Axel Uriel 319142563

Fecha de entrega: 06/04/2026

1. Introducción

En el ámbito del procesamiento digital de señales, la transición de la teoría a la implementación práctica requiere de herramientas que optimicen el uso de recursos de hardware y software. Esta sección establece la base teórica del reporte, diferenciando la operación matemática de su implementación algorítmica y justificando la necesidad de esta última.

1.1. Diferencia entre DFT y FFT

La **Transformada Discreta de Fourier (DFT)** es la herramienta fundamental para el análisis de señales en el dominio de la frecuencia. Matemáticamente, para una secuencia finita de N muestras, se define como:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \ldots, N-1 \tag{1}$$

Es importante precisar que la DFT es la *transformación* en sí misma. Por el contrario, la **Transformada Rápida de Fourier (FFT)** no es una nueva transformada, sino un *algoritmo eficiente* diseñado para calcular los resultados de la DFT reduciendo drásticamente el número de operaciones aritméticas necesarias.

1.2. La reducción de la complejidad computacional

La necesidad de utilizar la FFT surge de las limitaciones de la DFT directa cuando el número de muestras N aumenta.

- **DFT Directa:** Para calcular cada uno de los N coeficientes de frecuencia, se requieren N multiplicaciones complejas. Por lo tanto, la complejidad total es del orden de $O(N^2)$.
- **FFT (Radix-2):** Al aprovechar las propiedades de simetría y periodicidad de los factores de giro, el algoritmo descompone la DFT de N puntos en transformadas más pequeñas, logrando una complejidad de $O(N \log_2 N)$.

2. Planteamiento del problema

El objetivo central de este desarrollo es el cálculo eficiente de la Transformada Discreta de Fourier (DFT) para una secuencia de datos de longitud finita $N = 1024$. Matemáticamente, para una señal $x[n]$, la DFT se define mediante la siguiente expresión:

$$X[k] = \sum_{n=0}^{1023} x[n] W_{1024}^{kn}, \quad k = 0, 1, \ldots, 1023 \tag{2}$$

donde $W_{1024}^{kn} = e^{-j\frac{2\pi}{1024}kn}$ representa los factores de giro o núcleos de la transformación.

2.1. Identificación de la estructura Radix-2

Para la resolución de este problema, se identifica que el número de puntos es una potencia exacta de dos, específicamente $1024 = 2^{10}$. Esta propiedad es fundamental, ya que permite la implementación de un algoritmo FFT de tipo Radix-2 mediante la descomposición sucesiva de la DFT en etapas de menor orden. En consecuencia, el algoritmo se organiza en un total de **10 etapas** de procesamiento.

2.2. Análisis de la carga computacional

La elección de una FFT frente a una implementación directa de la DFT para $N = 1024$ se fundamenta en el ahorro sustancial de recursos:

1. **DFT directa:** Requeriría un total de $N^2 = 1024^2 = 1,048,576$ multiplicaciones complejas.
2. **FFT (10 etapas):** Al implementarse mediante mariposas, el costo se reduce a únicamente $\frac{N}{2}\log_2(N) = \frac{1024}{2}(10) = 5,120$ multiplicaciones complejas.

Este ahorro representa una mejora en el rendimiento superior al 99.5 %, permitiendo el procesamiento de señales en tiempo real que de otro modo sería computacionalmente inviable bajo las restricciones de hardware comunes en ingeniería.

3. Desarrollo por mariposas

La Transformada Rápida de Fourier (FFT) fundamenta su eficiencia en la descomposición de la DFT en operaciones de menor orden mediante la estructura conocida como "mariposa" (butterfly). En esta sección se detalla la unidad básica de cómputo y el flujo de información asociado.

3.1. Descripción de la operación básica de mariposa

En un algoritmo Radix-2, la mariposa es el bloque elemental que procesa dos entradas complejas (A y B) para producir dos salidas (X e Y). En un esquema de decimación en el tiempo, la operación se rige por las siguientes ecuaciones lineales:

$$X = A + W_N^k B \tag{3}$$

$$Y = A - W_N^k B \tag{4}$$

Esta estructura permite que una sola multiplicación compleja y dos sumas generen dos puntos de la transformada, optimizando el uso de recursos aritméticos.

3.2. Factores de giro (Twiddle Factors)

El factor de giro W_N^k es un coeficiente complejo que representa la rotación en el plano unitario necesaria para la combinación de las muestras. Su definición matemática es:

$$W_N^k = e^{-j\frac{2\pi k}{N}} = \cos\left(\frac{2\pi k}{N}\right) - j\sin\left(\frac{2\pi k}{N}\right) \tag{5}$$

Para una FFT de $N = 1024$, el exponente k varía en cada etapa siguiendo la lógica de la descomposición del algoritmo.

3.3. Agrupamiento de muestras y flujo de datos

El flujo de datos describe la trayectoria de las muestras desde la entrada $x[n]$ hasta la salida $X[k]$. En cada una de las 10 etapas, las muestras se agrupan de la siguiente manera:

- **Agrupación:** Las muestras se combinan en parejas que forman una mariposa. En la primera etapa, las muestras se agrupan de dos en dos, mientras que en etapas superiores se combinan resultados de bloques previos.
- **Distancia de índices:** La separación entre los índices de A y B que alimentan una mariposa cambia en cada nivel. En la etapa $m = 1$, la distancia es de 1 muestra; para la etapa final ($m = 10$), la distancia es de 512 muestras.

3.4. Análisis a escala: FFT de 16 puntos

Para entender la estructura, se analizará un sistema de $N = 16$, el cual requiere $\log_2(16) = 4$ etapas.

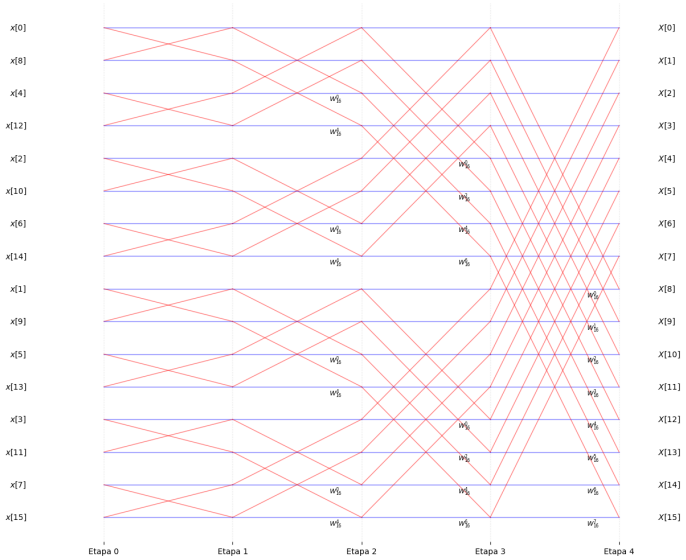

Figura 1: Diagrama de flujo para una FFT de 16 puntos con decimación en tiempo.

Como se observa en la Figura 1, las entradas ingresan en orden de inversión de bits para un sistema de $N = 16$: $x[0], x[8], x[4], x[12], x[2], x[10], x[6], x[14], x[1], x[9], x[5], x[13], x[3], x[11], x[7], x[15]$. Para la primera etapa (Stage 0), las ecuaciones de la primera mariposa se calculan utilizando el factor de giro correspondiente:

$$X_1[0] = x[0] + W_{16}^0 x[8] \tag{6}$$

$$X_1[1] = x[0] - W_{16}^0 x[8] \tag{7}$$

Este proceso de combinación de pares se repite para las 8 mariposas de la primera etapa, propagando los resultados intermedios hacia las etapas 2, 3 y 4 mediante la estructura de red (lattice). Finalmente, tras completar los 4 niveles de procesamiento, se obtienen los coeficientes de la transformada $X[k]$ en su orden natural de frecuencia.

4. Estructura de las 10 etapas

Al tratarse de una FFT de $N = 1024$ puntos basada en un algoritmo Radix-2, el proceso se descompone en exactamente $M = \log_2(1024) = 10$ etapas consecutivas de procesamiento . En esta sección se detalla la organización interna y la evolución de los datos en cada nivel.

4.1. Organización y número de operaciones

Cada una de las 10 etapas procesa la totalidad de las 1024 muestras. Dado que cada operación de mariposa toma dos entradas para producir dos salidas, el número de mariposas por etapa es constante e igual a:

$$\text{Mariposas por etapa} = \frac{N}{2} = \frac{1024}{2} = 512 \text{ mariposas} \tag{8}$$

Multiplicando esto por las 10 etapas del algoritmo, se obtiene un total de 5,120 operaciones de mariposa para completar la transformada.

4.2. Variación de la distancia entre índices

Una característica crítica en la implementación de la FFT es cómo cambia la distancia o el salto entre los índices de las muestras que alimentan a una misma mariposa en cada nivel. Esta distancia evoluciona de forma potencial según la etapa m (donde $m = 1, 2, \ldots, 10$):

- **Etapa 1 ($m = 1$):** Las mariposas operan sobre muestras adyacentes. La distancia es $2^{m-1} = 2^0 = 1$. Se combinan pares como $(x[0], x[1])$, $(x[2], x[3])$, etc.
- **Etapas intermedias:** La distancia se duplica en cada nivel. Por ejemplo, en la etapa 2 la distancia es 2, y en la etapa 5 la distancia es 16.
- **Etapa 10 ($m = 10$):** En el último nivel, la distancia alcanza su máximo valor: $2^{10-1} = 512$. Aquí se combinan muestras de la primera mitad del bloque con muestras de la segunda mitad (ej. $x[0]$ con $x[512]$).

Esta estructura de saltos crecientes permite que la información de cada muestra individual se propague y combine con todas las demás, logrando finalmente la resolución espectral completa de la señal.

5. Ordenamiento de datos

Para que una FFT de tipo Radix-2 con decimación en el tiempo (DIT) produzca resultados en el orden natural de frecuencia ($k = 0, 1, \ldots, N-1$), es imperativo realizar un pre-procesamiento de las muestras de entrada $x[n]$. Este proceso se conoce como reordenamiento por inversión de bits o *bit-reversal*.

5.1. Inversión de bits (Bit-reversal)

La lógica de este ordenamiento consiste en representar el índice decimal de cada muestra en su equivalente binario, invertir el orden de dichos bits y convertir el resultado de vuelta al sistema decimal. Dado que la FFT es de $N = 1024$ puntos, se identifica que $1024 = 2^{10}$, por lo que cada índice n se representa mediante una palabra de 10 bits.

5.2. Ejemplos de reordenamiento ($N = 16$)

A continuación, se ilustra el proceso para los primeros índices de la secuencia utilizando una representación de 4 bits, necesaria para el ejemplo de 16 puntos presentado anteriormente:

Índice Original (n)	**Binario**	**Bits Invertidos**	**Nuevo Índice**
0	0000	0000	0
1	0001	1000	8
2	0010	0100	4
3	0011	1100	12
4	0100	0010	2
5	0101	1010	10

Cuadro 1: Lógica de inversión de bits para un sistema de $N = 16$ puntos.

5.3. Importancia en la estructura del algoritmo

Si se optara por un esquema de decimación en frecuencia (DIF), el reordenamiento aparecería en la salida del algoritmo en lugar de la entrada. En el esquema propuesto de decimación en tiempo, el bit-reversal asegura que:

- Las muestras se agrupen correctamente para las operaciones de mariposa en la primera etapa.
- La complejidad de interconexión se mantenga manejable a lo largo de las 10 etapas.
- El resultado final $X[k]$ se obtenga de forma secuencial y ordenada.

6. Diagrama o esquema

Dada la magnitud de una FFT de 1024 puntos, la representación exhaustiva de cada una de las 1024 líneas de datos resulta impractical y visualmente saturada. No obstante, en esta sección se presenta la arquitectura general y un detalle técnico del flujo de señales.

6.1. Estructura general por etapas

El flujo general del algoritmo se presenta en la Figura 2. En este diagrama modular se observa la progresión de los datos desde la entrada $x[n]$ hasta la obtención de los coeficientes de frecuencia $X[k]$.

Como se aprecia en el esquema, el procesamiento se divide en tres fases fundamentales: el reordenamiento inicial por inversión de bits, las diez etapas de mariposas Radix-2 y la salida final en orden natural.

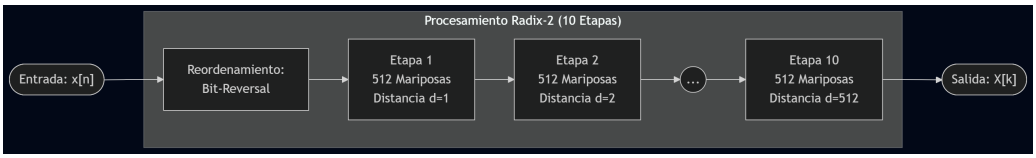

Figura 2: Diagrama de bloques funcional del algoritmo FFT de 1024 puntos con 10 etapas de procesamiento.

6.2. Escalamiento y detalle del algoritmo ($N = 1024$)

Para validar la lógica de interconexión en el sistema de 1024 puntos, se presenta en la Figura 3 un detalle estructural que abarca las primeras 32 muestras (índices 0-31) a lo largo de las 10 etapas de descomposición.

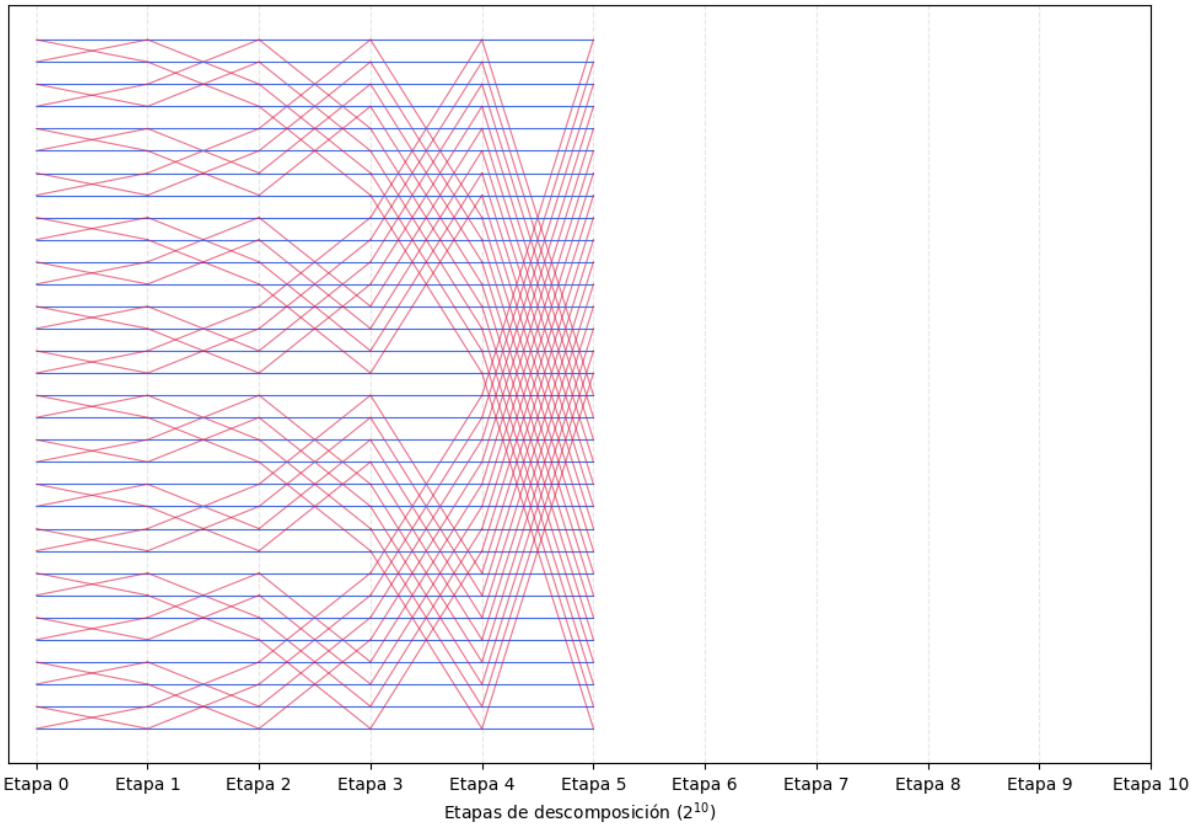

Figura 3: Detalle estructural de la interconexión de mariposas para $N = 1024$ (muestras 0-31).

Como se observa en el diagrama generado, el patrón de entrelazado escala de la siguiente forma para cubrir la totalidad del bloque:

- **Entrada:** 1024 datos reordenados mediante inversión de bits (bit-reversal).
- **Cuerpo:** 10 niveles de columnas de mariposas. Cada columna procesa 512 unidades de cómputo en paralelo.
- **Salida:** Coeficientes de frecuencia $X[k]$ obtenidos en orden natural tras la décima etapa.

7. Complejidad Computacional

El valor fundamental del algoritmo FFT Radix-2 radica en la drástica reducción de operaciones aritméticas necesarias para transformar una señal al dominio de la frecuencia. A continuación, se presenta un análisis comparativo para el caso de $N = 1024$ puntos.

7.1. Comparación entre el costo de una DFT directa y el costo de la FFT

El cálculo de la DFT mediante la sumatoria tradicional requiere realizar N multiplicaciones y sumas complejas por cada uno de los N coeficientes de salida. Por lo tanto, la complejidad es de orden cuadrático $O(N^2)$. Para $N = 1024$:

- **Multiplicaciones complejas:** $N^2 = 1,024^2 = 1,048,576$.
- **Adiciones complejas:** $N(N-1) = 1,024 \times 1,023 = 1,047,552$.

Al emplear la decimación en tiempo y la estructura de mariposas, la complejidad se reduce a un orden logarítmico $O(N \log_2 N)$. Dado que el sistema consta de 10 etapas y en cada una se realizan $N/2$ operaciones de mariposa:

- **Multiplicaciones complejas:** $\frac{N}{2} \log_2 N = 512 \times 10 = 5,120$.
- **Adiciones complejas:** $N \log_2 N = 1,024 \times 10 = 10,240$.

7.2. Análisis de Eficiencia

La eficiencia del algoritmo se puede visualizar mediante el factor de reducción de cómputo para las multiplicaciones:

$$\text{Factor de mejora} = \frac{N^2}{\frac{N}{2} \log_2 N} = \frac{1,048,576}{5,120} \approx 204{,}8 \tag{9}$$

Esto significa que para un sistema de 1024 puntos, la FFT es aproximadamente **205 veces más rápida** que la DFT directa. Este ahorro del 99.5 % en el procesamiento es lo que permite que el análisis espectral sea ejecutable en tiempo real en dispositivos con recursos limitados.

8. Conclusiones

8.1. Importancia de la FFT en el procesamiento de señales

Más allá de la optimización, la FFT es el motor que permite transitar del análisis puramente temporal al análisis en frecuencia de forma práctica. Se infiere que este algoritmo no solo acelera procesos, sino que habilita el diseño de sistemas complejos (como radares o sistemas de comunicación 5G) que serían físicamente imposibles de operar bajo la estructura de una DFT directa, transformando una barrera matemática en una herramienta de ingeniería funcional.

8.2. Viabilidad de implementación: Software vs. Hardware

La validación realizada en software demuestra que la FFT es sumamente flexible y fácil de desplegar en procesadores de propósito general para tareas de diagnóstico y análisis no crítico. Sin embargo, dada la independencia de las operaciones de mariposa en cada nivel, se concluye que la verdadera potencia del algoritmo reside en su capacidad de paralelismo. Mientras que el software es ideal por su flexibilidad, el hardware dedicado (como FPGAs o DSPs) es la única vía viable para aplicaciones de ultra-baja latencia, donde las etapas del algoritmo pueden ejecutarse de forma concurrente para procesar flujos de datos masivos en tiempo real.

Referencias

[1] Juli. *Cuadro Comparativo Tf*. Scribd. Recuperado de: `https://www.scribd.com/document/415105199/Cuadro-Comparativo-Tf-Juli`

[2] Wikipedia contributors. (2026). *Cooley–Tukey FFT algorithm*. Wikipedia, The Free Encyclopedia. Recuperado de: `https://en.wikipedia.org/wiki/Cooley%E2%80%93Tukey_FFT_algorithm`